

Exercise sheet 2 — Solutions

Data – values, variables, data types and expressions

Published after the in-class discussion. Read these only once you have tried the sheet.

Part A — Values, variables and types

A1 Value and type

	expression	value	type
1	<code>7 + 3</code>	<code>10</code>	<code>int</code>
2	<code>7 / 1</code>	<code>7.0</code>	<code>float</code> — <code>/</code> always gives a float
3	<code>2 ** 3 ** 2</code>	<code>512</code>	<code>int</code> — <code>**</code> groups right to left
4	<code>7 // 2 + 7 % 2</code>	<code>4</code>	<code>int</code> — <code>3 + 1</code>
5	<code>-9 // 4</code>	<code>-3</code>	<code>int</code> — rounds down, so $-2.25 \rightarrow -3$
6	<code>8 / 4 * 2</code>	<code>4.0</code>	<code>float</code> — left to right, not division last
7	<code>"7" + "3"</code>	<code>"73"</code>	<code>str</code> — <code>+</code> joins text
8	<code>3 * "ab"</code>	<code>"ababab"</code>	<code>str</code> — <code>*</code> repeats text
9	<code>10 % 3 == 1</code>	<code>True</code>	<code>bool</code>
10	<code>int("7") + 3</code>	<code>10</code>	<code>int</code>
11	<code>float(7)</code>	<code>7.0</code>	<code>float</code>
12	<code>int(-3.9)</code>	<code>-3</code>	<code>int</code> — chops towards zero

The two that might catch you are **3** and **5**, closely followed by **6**.

Worth noticing. Numbers 5 and 12 disagree, and that is not a mistake. `//` rounds *down* (towards $-\infty$), so `-9//4` is `-3` from -2.25 . `int()` *chops* (towards zero), so `int(-3.9)` is `-3`. For positive numbers they agree, while for negative ones they do not.

A2 Trace it

a)

line	a	b
<code>a = 3</code>	<code>3</code>	—
<code>b = 8</code>	<code>3</code>	<code>8</code>
<code>a = a + b</code>	<code>11</code>	<code>8</code> <code>3 + 8</code>
<code>b = b - a</code>	<code>11</code>	<code>-3</code> <code>8 - 11</code>
<code>a = a * 2</code>	<code>22</code>	<code>-3</code>
<code>b = a % b</code>	<code>22</code>	<code>-2</code> <code>22 mod -3</code>

b) a is 22, b is -2 .

The last line is worth pausing on. $22 \% -3$ is -2 , not 1: in Python the result of $\%$ takes the sign of the *right-hand* operand. Anyone who wrote 1 was applying the school definition of remainder, which is a perfectly reasonable thing to expect and simply is not what Python does.

c) `ZeroDivisionError`. It is a fourth name, and you have not met it — any answer that recognises dividing by zero is the problem is correct here.

A3 *Legal, and good*

name	legal?	good?	
<code>total_price</code>	yes	yes	
<code>class</code>	no	—	a Python keyword
<code>Price</code>	yes	no	capital initial is reserved for classes by convention
<code>2ndAttempt</code>	no	—	starts with a digit
<code>price_in_euros</code>	yes	yes	
<code>my variable</code>	no	—	contains a space
<code>x</code>	yes	depends	fine in a two-line calculation, poor in a program
<code>thing</code>	yes	no	legal, but tells the reader nothing
<code>l</code>	yes	no	see below
<code>TotalPrice</code>	yes	no	Python convention is <code>total_price</code>
<code>n_students</code>	yes	yes	
<code>temperature_c</code>	yes	yes	the unit in the name is a good habit

The one with a non-style problem is `l`. In most fonts a lower-case L is indistinguishable from the digit 1 and near enough to a capital I. A bug hidden by your choice of font is a bug you will not find by reading.

A4 *Brackets in the wrong place*

fix	what the broken version did
<code>average = (a + b + c) / 3</code>	added a, b and one third of c
<code>area = PI * r ** 2</code>	computed the circumference, $2\pi r$
<code>half = (x + y) / 2</code>	computed $x/2 + y$
<code>total = n * p * (1 + 0.22)</code>	computed $np + 0.22$ — adding 22 cents, not 22 per cent

Worth noticing. Every one of these runs gives a plausible-looking number. That is the point of the exercise: some errors do not announce themselves. The secure way is to put brackets in whenever you are not certain, and to check the answer against a case you can do in your head.

A5 *Negative numbers*

a, b) $-7 // 2$ is -4 and $-7 \% 2$ is 1.

c) $-7/2$ is -3.5 . Rounding *down* means going to the nearest whole number below -3.5 , which is -4 . Rounding towards zero would give -3 — that is what `int(-3.5)` does, and it is a different operation.

d) From $(a // b) * b + (a \% b) == a$ with $a = -7$, $b = 2$: $(-4) \times 2 + r = -7$, so $r = 1$.

Worth noticing. The identity in (d) is the actual **definition**. `//` and `%` are not two independent operations — fixing one determines the other. This is also why `%` takes the sign of the right operand, which explains A2(b).

A6 *The one from last week*

a) A float is stored in a fixed number of bits. One tenth in binary is a non-terminating fraction, exactly as one third is in decimal, so Python stores the closest value it can. The tiny error is invisible on its own and becomes visible when several such values are combined.

b)

```
abs(0.1 + 0.2 - 0.3) < 1e-9
```

Any tolerance-based test is correct.

c) No. An `int` is stored exactly, and Python integers have no size limit, so `a == b` is always trustworthy for integers. This is a genuine reason to prefer `int` when you can — for example, count money in cents rather than in euros.

Part B — Writing small programs

B1 *A circle*

```
# the constants and the radius
# the area, from pi r squared
# the circumference, from 2 pi r
# print both, labelled

PI = 3.14159
radius = 3.5

area = PI * radius ** 2
circumference = 2 * PI * radius

print("area:", area)
print("circumference:", circumference)
```

Changing the radius means editing line 2 and nothing else.

B2 *A shopping total*

```
# how much the notebooks cost
# how much the pens cost
# the total before VAT
# the VAT on that total
# the total to pay

VAT_RATE = 0.22

notebooks = 3 * 2.40
pens = 2 * 1.15

net = notebooks + pens
vat = net * VAT_RATE
gross = net + vat

print("before VAT:", net)
print("VAT:", vat)
print("to pay:", gross)
```

Output: 9.5, 2.09, 11.59 — or something ending in a long tail of digits, depending on the arithmetic. Either is fine this week; we cannot format it properly until lecture 3.

B3 *Seconds, again*

```
# whole hours, and the seconds left over
# whole minutes from what is left
# the seconds that remain
# print it

total_seconds = 10000

hours = total_seconds // 3600
```

```
rest = total_seconds % 3600

minutes = rest // 60
seconds = rest % 60

print(hours, "h", minutes, "m", seconds, "s")
```

Output: 2 h 46 m 40 s.

The order is what makes it four lines. Taking `minutes = total_seconds // 60` first gives 166 minutes, which is correct and useless — the hours have not been removed.

B4 Swap

Without a third variable:

```
a = 12
b = 5

a = a + b      # a is 17
b = a - b      # 17 - 5 = 12
a = a - b      # 17 - 12 = 5
```

With one:

```
temp = a
a = b
b = temp
```

The first version is a puzzle, while the second states what is happening and is better. Cleverness that costs the reader thirty seconds is not a saving.

Worth noticing. Python has a third way — `a, b = b, a` — which is both short and clear. That is the one to use, and we meet it properly in lecture 7.

B5 What does it print?

	prints	why
1	5	<code>y</code> took a copy of the value 5; changing <code>a</code> later does not reach it
2	20	$7 + 3 = 10$, then 10×2
3	5.0	<code>/</code> always gives a float
4	205	both are <code>str</code> , so <code>+</code> joins rather than adds
5	False True True	three comparisons, three booleans

Worth noticing. Number 1 matters more than it looks. `y = x` attaches `y` to the *value* `x` currently has — it does not tie the two names together. Nothing you do to `x` afterwards affects `y`. Keep this in mind: in lecture 6 we meet a case where the answer is different, and knowing why is the whole point of that lecture.

B6 *Three broken programs*

	error	why	fix
1	TypeError	age is a str; you cannot add an int to it	next_year = int(age) + 1
2	NameError	count was never given a value	give it one before line 2
3	ValueError	"seven" is a str, as int() wants, but it does not contain digits	n = 7, or use "7"

One, two, three — the three error names from the lecture, in the same order.

B7 *It runs, and it is wrong*

a) average: 66.33333333333333

b) 27.666666666666668

c)

```
average = (mark1 + mark2 + mark3) / 3
```

d) Because nothing tells you. An error stops the program and points at a line; a wrong answer looks exactly like a right one and propagates into everything downstream. The only protection is checking against a case you can compute in your head, and reading your own code as if someone else wrote it. In larger code bases, these kinds of checks can also be automated with **unit tests**.

Worth noticing. A mark of 66 out of 30 is impossible, and that is the tell. Get into the habit of asking whether the number you got is *plausible* before asking whether it is right — it catches most of these in a second.

If you finish early

Changing the VAT rate should touch one line, because it was written as VAT_RATE = 0.22 at the top.

Adding a second product means another two lines and another term in the sum. Adding a hundred means a hundred. That is exactly the wall lecture 4 (loops) and lecture 6 (lists) exist to knock down, and it is worth saying so now, while the frustration is fresh.